

# DevPilot Complete Learning Guide

---

**Project:** DevPilot

**Version in this archive:** 0.3.0

**Purpose:** A local, approval-gated AI debugging workbench for small Python repositories.

---

## 1. What This Project Is

---

DevPilot is a local engineering tool that helps investigate software bugs. It copies a repository into a disposable workspace, lets an agent inspect files and run tests, asks the human before risky actions, records everything that happened, and exports evidence such as a patch, report, and trace.

Think of it as a controlled debugging assistant with a browser UI.

It is not a production SaaS, not a multi-user platform, not a free-form terminal agent, and not a tool that automatically pushes code to GitHub. The core design idea is simple:

> AI can suggest actions, but the application enforces the safety boundary.

DevPilot is built for one operator at a time. It is most suitable for small Python projects that use `pytest`.

---

## 2. What Problem It Solves

---

Normal AI coding tools can make mistakes, skip evidence, or claim something is fixed without proving it. DevPilot tries to solve that by connecting every step to recorded evidence.

It can:

- Inspect a repository copy.
- Search code and documentation.
- Read files.

- Ask to run tests.
- Ask before editing files.
- Generate a Git patch.
- Record a trace of every important action.
- Mark a run verified only when tests pass on the final approved snapshot.

The important part is that verification is not just the model saying “done.” Verification comes from tool results, test exit codes, and snapshot identity.

---

### 3. What It Does In One Demo Run

---

A typical demo run looks like this:

1. You open the browser UI.
2. You paste the local access token.
3. You select a demo repository such as `demo-redis`.
4. You click **Investigate**.
5. DevPilot copies the demo repo into a disposable run directory.
6. The scripted demo agent lists/searches/reads files.
7. It asks for approval before running tests.
8. You approve.
9. It runs the fixed pytest command.
10. It asks for approval before applying a patch.
11. You approve.
12. It edits only the copied workspace.
13. It asks to run tests again.
14. You approve.
15. It records whether tests passed.
16. It exports a patch, report, and JSON trace.

Your original source repository is not edited directly.

---

## 4. Current Local Run Command

---

For this machine, the easiest way to run is:

```
cd "/home/cunam/Music/TMPFolder/Project/devpilot-devops-complete- (2)/devpilot"
./run.sh
```

Then open:

```
http://127.0.0.1:8091
```

The script prints the token. Paste it into the UI and click **Connect**.

To stop:

```
./stop.sh
```

If setup has not been done yet:

```
./setup.sh
./run.sh
```

---

## 5. Important Local Files

---

The main project folder contains these important files and directories:

| Path            | Meaning                                                        |
|-----------------|----------------------------------------------------------------|
| `README.md`     | Main project overview and official start instructions          |
| `QUICKSTART.md` | Short local instructions added for easy learning               |
| `setup.sh`      | Creates config, virtual environment, and installs dependencies |
| `run.sh`        | Starts the app, prints URL and token                           |
| `stop.sh`       | Stops the app started by `run.sh`                              |

|                                  |                                                                  |
|----------------------------------|------------------------------------------------------------------|
| <code>`.env`</code>              | Local runtime configuration                                      |
| <code>`.secrets/`</code>         | Generated local tokens                                           |
| <code>`.venv/`</code>            | Python virtual environment                                       |
| <code>`.devpilot-local/`</code>  | Local app state used by <code>`run.sh`</code>                    |
| <code>`.workspace/`</code>       | Repositories available to the app                                |
| <code>`.devpilot/`</code>        | Main Python application package                                  |
| <code>`.devpilot/static/`</code> | Browser UI HTML, CSS, and JavaScript                             |
| <code>`.devpilot/mcp/`</code>    | Minimal MCP client/server implementation                         |
| <code>`.devpilot/tools/`</code>  | Tool implementations and runner adapters                         |
| <code>`.docs/`</code>            | Architecture, API, operations, model, Kubernetes, and other docs |
| <code>`.tests/`</code>           | Project test suite                                               |
| <code>`.compose.yaml`</code>     | Docker Compose demo path                                         |
| <code>`.helm/`</code>            | Kubernetes Helm chart                                            |
| <code>`.monitoring/`</code>      | Prometheus, Grafana, Alertmanager, Tempo config                  |

---

## 6. Technology Stack

---

### Backend

- **Python 3.11+ / 3.12** on this machine
- **FastAPI** for HTTP API and static UI serving
- **Uvicorn** as the local ASGI server

- **Pydantic Settings** for environment-based configuration
- **SQLite** for local audit/history state
- **pytest** for repository test execution

## Frontend

- Plain **HTML**
- Plain **CSS**
- Plain **JavaScript**
- No React/Vue build pipeline
- Browser `fetch` API for backend calls
- Server-Sent Events for streaming trace events

## Agent / Model

Two provider modes exist:

| Provider              | Meaning                                                                          |
|-----------------------|----------------------------------------------------------------------------------|
| <code>`demo`</code>   | Scripted replay. No language model is called. Useful for safe demos.             |
| <code>`openai`</code> | OpenAI-compatible local inference endpoint, usually Ollama or vLLM running Qwen. |

The name `openai` here means “OpenAI-compatible API format,” not necessarily OpenAI cloud.

## Tools / Protocol

- Minimal **MCP 2025-06-18 subset**
- JSON-RPC over stdio
- Tool servers for files, Git, docs/search, terminal tests, and database query

## DevOps / Deployment

- Docker and Docker Compose

- Optional monitoring stack with Prometheus/Grafana/Alertmanager/Tempo
  - Kubernetes manifests and Helm chart
  - Argo CD examples
  - Terraform helper for existing clusters
- 

## 7. Why These Technologies Were Chosen

---

FastAPI is used because the app needs a small, clear HTTP API and browser UI. SQLite is used because the app is single-operator and local/private, so a full database server would add complexity without solving the main problem. MCP-style tools provide a clean boundary between the model and actions. Docker/Kubernetes runner options exist because running tests means executing project code, and project code should be isolated when it is not fully trusted.

The frontend is intentionally simple. Since the app is an operational workbench, plain HTML/CSS/JS keeps the UI easy to inspect and avoids a heavy frontend build process.

The approval system exists because prompts are not security boundaries. The model may request a tool, but the app decides whether that request is allowed and whether human approval is required.

---

## 8. Main Runtime Modes

---

### Demo Mode

Demo mode uses predefined scripted actions. It is safe for learning the workflow because it does not require a real LLM.

Current local mode:

```
Provider: demo
runner: host-trusted
```

This means the model behavior is scripted, and tests run on the host. Use this only with the bundled trusted demo repositories.

## Real Model Mode

Real model mode connects to an OpenAI-compatible endpoint such as Ollama or vLLM.

Typical settings:

```
UP_PROVIDER=openai
UP_BASE_URL=http://127.0.0.1:11434/v1
UP_MODEL=qwen3:4t
UP_LLM_API_KEY=local
UP_RUNNER=docker
```

In real model mode, Docker or Kubernetes runner isolation is recommended.

## Docker Mode

Docker Compose is supported:

```
python3 scripts/bootstrap.py
docker compose up -d --build
```

On this machine Docker built the image but failed while starting the container with:

```
operation not permitted
```

That is most likely a local Docker/security/runtime restriction, not an application bug. The host Python path avoids it.

## Kubernetes Mode

Kubernetes mode runs tests in separate Kubernetes Jobs. This is the strongest shipped isolation path but requires a working cluster, RBAC, namespace policy, and runner image.

---

## 9. How The Browser UI Works

The UI is served from:

```
devpilot/static/index.html
```

The browser flow is:

1. User pastes token.
2. `app.js` calls `GET /api/config`.
3. It stores the token in browser session storage.
4. It loads repositories using `GET /api/repos`.
5. User clicks **Investigate**.
6. UI sends `POST /api/runs`.
7. UI streams events from `GET /api/runs/{id}/events`.
8. UI polls `GET /api/runs/{id}` for status and approvals.
9. If approval is needed, UI shows **Approve this action** and **Deny**.
10. User decision is sent to `POST /api/runs/{id}/approvals/{approval\_id}`.
11. When complete, UI enables artifact downloads.

The UI does not execute code. It only calls backend APIs.

---

## 10. Main API Endpoints

---

| Endpoint          | Purpose                         |
|-------------------|---------------------------------|
| `GET /`           | Serves the web UI               |
| `GET /livez`      | Basic process health            |
| `GET /readyz`     | Readiness and drain state       |
| `GET /api/config` | Shows non-secret runtime config |
| `GET /api/repos`  | Lists allowed repositories      |
| `POST /api/runs`  | Starts a run                    |
| `GET /api/runs`   | Lists recent runs               |

|                                                            |                                                                                  |
|------------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>`GET /api/runs/{id}`</code>                          | Gets one run status                                                              |
| <code>`GET /api/runs/{id}/events`</code>                   | Streams trace events                                                             |
| <code>`POST /api/runs/{id}/approvals/{approval_id}`</code> | Approves or denies one action                                                    |
| <code>`POST /api/runs/{id}/cancel`</code>                  | Cancels active work                                                              |
| <code>`GET /api/runs/{id}/artifacts/{kind}`</code>         | Downloads <code>`patch`</code> , <code>`report`</code> , or <code>`trace`</code> |
| <code>`GET /metrics`</code>                                | Prometheus metrics with separate token                                           |

All `/api/*` calls require:

```
Authorization: Bearer <token>
```

## 11. Internal Backend Flow

The main backend flow is controlled by `devpilot/runtime.py`.

Important classes:

| Class/File                                             | Job                                            |
|--------------------------------------------------------|------------------------------------------------|
| <code>`Settings`</code> in <code>`config.py`</code>    | Reads and validates environment config         |
| <code>`RunManager`</code> in <code>`runtime.py`</code> | Owns active runs, approvals, lifecycle         |
| <code>`Store`</code> in <code>`store.py`</code>        | Writes SQLite audit/history data               |
| <code>`Gateway`</code> in <code>`mcp/client.py`</code> | Starts MCP subprocess servers and routes tools |
| <code>`DemoModel`</code> in <code>`models.py`</code>   | Scripted demo action sequence                  |

|                                               |                                          |
|-----------------------------------------------|------------------------------------------|
| <code>`OpenAIModel` in<br/>`models.py`</code> | Calls OpenAI-compatible chat completions |
| <code>`Workspace` in `safety.py`</code>       | Validates workspace paths and snapshots  |

Run lifecycle:

```

PUSH /api/run
  - create workspace
  -> create run record in S0Life
  - copy workspace contents to /tmp
  -> initialize git in copied workspace
  - create commit
  -> expose allowed MCP tool
  - model asks for tool arg
  -> app validates tool names and arguments
  -> risky tools request human approval
  -> approved tools execute
  -> results are sent back to model
  -> final status, verification, and artifacts are saved

```

## 12. MCP Tool System

MCP means Model Context Protocol. In this project it provides a structured tool boundary.

DevPilot starts subprocess servers like:

```

python -m devpilot.mcp.server_files
python -m devpilot.mcp.server_git
python -m devpilot.mcp.server_docs
python -m devpilot.mcp.server_termina
python -m devpilot.mcp.server_database

```

The model sees namespaced tool names such as:

```

files_read_file
files_replace_text
docs_search

```

```
git diff
terminal run tests
database ques
```

The model cannot run arbitrary shell commands. The terminal server exposes a fixed test command workflow, not an unrestricted terminal.

---

## 13. Approval System

---

The approval system is one of the most important parts of DevPilot.

Actions with risk require approval:

- File edits
- Test execution

When approval is required, DevPilot records:

- Tool name
- Arguments
- Argument hash
- Workspace snapshot hash
- Preview diff, if it is an edit
- Expiry time
- Warning message

The UI shows a human checkpoint. The user can approve or deny.

Approvals are:

- Single-use
- Bound to one run
- Bound to one action
- Bound to one workspace snapshot
- Rejected if stale or already answered

This prevents the model from obtaining broad permission.

---

## 14. Verification

---

DevPilot separates claims from evidence.

The model can write a final explanation, but the `verified` field comes from the application.

A run is verified only when:

- Tests were executed.
- The final snapshot matches the tested snapshot.
- The last relevant test exit code is zero.

This does not prove the whole project is perfect. It only proves the selected tests passed on that snapshot.

---

## 15. Data Storage

---

DevPilot uses SQLite for local audit/history state.

Typical local script state:



The database stores:

- Runs
- Events
- Pending approval data
- Final summary

- Verification state
- Metrics

Run artifacts are generated from stored state and workspace diffs.

---

## 16. Workspace Safety

---

The original repo is not edited directly. DevPilot copies the selected repo into a run directory.

Example:

```
workspace/demo-redis      original demo repo
devpilot-local/runs/<id>/workspace  copied run workspace
```

Edits happen in the copied run workspace. At the end, DevPilot produces a patch that you can review.

Repository names are validated. Paths are checked to avoid using arbitrary file paths or symlinks as selected repositories.

---

## 17. Runner Options

---

| Runner                      | What it does                    | When to use                           |
|-----------------------------|---------------------------------|---------------------------------------|
| <code>`disabled`</code>     | Does not run tests              | Safe inspection only                  |
| <code>`host-trusted`</code> | Runs tests on this computer     | Only bundled trusted demos            |
| <code>`docker`</code>       | Runs tests in a local container | Real local projects when Docker works |
| <code>`kubernetes`</code>   | Runs tests as Kubernetes Jobs   | Stronger cluster-based isolation      |

On this machine, Docker had a runtime permission problem, so `host-trusted` is used for the bundled

demo.

---

## 18. Security Model

---

Trusted:

- Local operator
- DevPilot backend
- Tool implementation code
- Runner configuration
- Local model endpoint configured by the operator

Untrusted:

- Model text
- Repository contents
- Test output
- Any requested tool arguments from the model

Key protections:

- Explicit token authentication
  - Separate metrics token
  - Allowed hosts validation
  - No arbitrary shell tool
  - Approval-gated writes and tests
  - Disposable workspace copies
  - Snapshot hashes
  - Single active run
  - Local state lock
  - No automatic push or merge
-

## 19. Observability

---

DevPilot can expose operational metrics for Prometheus and traces through OpenTelemetry.

The monitoring stack includes:

- Prometheus
- Grafana
- Alertmanager
- Tempo
- OpenTelemetry Collector

Metrics are about operational behavior, such as:

- Request rate
- Latency
- Tool errors
- Run outcomes
- Approval waits
- Model timing
- Token usage when returned

The metrics endpoint requires a separate metrics token.

---

## 20. DevOps And Deployment Pieces

---

The project includes several deployment-related folders:

| Folder                  | Purpose                     |
|-------------------------|-----------------------------|
| <code>`.github/`</code> | GitHub Actions workflows    |
| <code>`.helm/`</code>   | Helm chart for Kubernetes   |
| <code>`.argocd/`</code> | Argo CD example app/project |

|                            |                                          |
|----------------------------|------------------------------------------|
| <code>`infra/`</code>      | Kind/Kubernetes helper manifests         |
| <code>`terraform/`</code>  | Optional installer for existing clusters |
| <code>`monitoring/`</code> | Metrics/tracing/dashboard config         |

The architecture intentionally uses one app replica. This is because approvals and active task ownership are in memory, while history is SQLite. Multiple replicas could disagree about the active approval state.

High availability would require a larger redesign: shared database, durable queue, shared approval store, artifact store, and ownership model.

## 21. Demo Repositories

The bundled demo repositories are seeded into `workspace/`.

Examples:

| Demo                           | What it demonstrates                 |
|--------------------------------|--------------------------------------|
| <code>`demo-redis`</code>      | Configuration bug repair             |
| <code>`demo-pagination`</code> | Off-by-one pagination bug            |
| <code>`demo-normalize`</code>  | Unicode/whitespace normalization bug |

In demo mode the sequence is scripted. It proves the integration workflow, not model intelligence.

## 22. What Happened In Your Browser Screenshot

Your screenshot showed:

```
workspace: connected
Provider: Scripted repla
Runner: host-trusted
Repository: demo-redis
Status: Awaiting approval
Tool: terminal run test
```

That means:

1. The UI connected successfully.
2. The backend loaded the demo repo.
3. A run started.
4. The demo agent reached the test-running step.
5. DevPilot paused because tests execute code.
6. It asked you to approve `terminal\_\_run\_tests`.

For the bundled demo, approving that action is expected.

---

## 23. How To Use It Step By Step

---

### Start

```
cd ~/home/punam/music/InFolder/Project/devpilot-devops-complete (2)/devpilot
./run.sh
```

### Open

```
http://127.0.0.1:8091
```

### Connect

Paste the token printed by `run.sh`.

## Run Demo

1. Select `demo-redis`.
2. Click **Redis configuration** or use the prefilled task.
3. Click **Investigate**.
4. Approve test execution.
5. Approve the patch if shown.
6. Approve final test execution.
7. Review the summary.
8. Download patch/report/trace.

## Stop



## 24. Common Problems

---

### Token rejected

Use the token printed by `./run.sh`. Do not use the metrics or Grafana token.

### Port already used

Run on another port:



### Already running

Stop it:



## Docker says operation not permitted

That is a local Docker/runtime permission issue. Use:



instead of Docker Compose.

## Approval screen appears

That is normal. DevPilot is waiting for your permission before doing something risky.

---

## 25. How To Explain This Project In An Interview

---

You can say:

> DevPilot is a local AI debugging workbench built with FastAPI, SQLite, a browser UI, and MCP-style tool subprocesses. It investigates a copied repository, lets an agent request structured tools, gates risky actions through explicit human approval, runs tests through configured runners, records an audit trail, and exports patch/report/trace artifacts. It separates model claims from verification by deriving verified status from test results and workspace snapshot identity.

Short version:

> It is a safe local AI debugging tool that proves fixes with tests and human-approved actions.

---

## 26. What Is Complete And What Is Not

---

Complete enough for demo/portfolio:

- Local UI
- Token auth
- Demo repos
- Scripted workflow
- Approval checkpoints

- Test execution
- SQLite audit history
- Patch/report/trace artifacts
- Docker/Kubernetes/monitoring docs and configs
- Improved local quickstart scripts
- Professional UI polish

Not complete as a production SaaS:

- No multi-user login system
  - No horizontal scaling
  - No hosted cloud service
  - No automatic GitHub PR creation
  - No unrestricted shell
  - No production-grade hostile-code sandbox
  - No guaranteed live model repair accuracy
- 

## 27. Best Learning Path

---

1. Run `./run.sh`.
  2. Complete the `demo-redis` workflow.
  3. Download the patch and report.
  4. Open `devpilot/static/app.js` to understand the UI flow.
  5. Open `devpilot/api.py` to understand routes.
  6. Open `devpilot/runtime.py` to understand run lifecycle.
  7. Open `devpilot/models.py` to understand demo vs real model behavior.
  8. Open `devpilot/mcp/server.py` and `devpilot/mcp/client.py` to understand tools.
  9. Read `docs/ARCHITECTURE.md`.
  10. Try `demo-pagination` and `demo-normalize`.
-

## 28. Quick Command Reference

---

```
# Setup
# ...
# ...
# ...
# Run on a different port
# PORT=8092 ./run.sh
# Stop
# ./stop.sh
# ...
# ...
# Run tests for DevPilot itself
./venv/bin/python -m pytest
```

---

## 29. Final Summary

---

DevPilot is a practical project because it combines AI, tool execution, human approval, evidence capture, and DevOps thinking. Its strongest idea is not “AI writes code.” Its strongest idea is:

> Every important action should be controlled, recorded, and proven.

That is why the project has approvals, snapshot hashes, copied workspaces, test evidence, and downloadable artifacts.